

# **BCSE497J - Project-I**

## **Autonomous Quadplane UAV for Coconut Disease Detection**

|           |                                  |
|-----------|----------------------------------|
| 23BCE2032 | <b>U MONISH</b>                  |
| 23BCE2091 | <b>MOHAMAD RIYAZUDEEN NADEEM</b> |
| 23BCE2097 | <b>TAVISA TEJ MANI SHANKAR</b>   |

Under the Supervision of

**Dr. Naresh K**  
Designation

School of Computer Science and Engineering (SCOPE)

**B.Tech.**

*in*

**Computer Science and Engineering**

**School of Computer Science and Engineering**



September 2026

## ABSTRACT

Coconut palms across India lose significant yield to diseases — bud rot, stem bleeding, leaf rot and grey leaf spot — and to pests such as the rhinoceros beetle, red palm weevil and whitefly. These are manageable when caught early and very hard to reverse once established, but early detection depends on regularly inspecting individual palms, and manual ground-level inspection across a large plantation is slow, subjective and expert-dependent.

This project designs an autonomous quadplane UAV — vertical take-off and landing for launch and recovery, fixed-wing for efficient cruise — running ArduPilot QuadPlane firmware with an onboard RGB camera. It flies a two-tier scout-then-confirm mission over the plantation; imagery is recorded on board, a YOLO-based model detects disease and pest symptoms after landing, and a Python route planner keeps a per-tree record and drops cleared trees from later missions.

Project-I is a simulation proof of concept. A complete autonomous mission — VTOL take-off, transition, a survey grid and a VTOL landing — was flown in ArduPilot SITL with every flight-phase check passing. Real two- and three-dimensional aerodynamic analysis of the chosen SD7037 wing and NACA 0009 tail was ported to a JSBSim model that passed all eight power-off stability checks, and an ArduPilot-to-JSBSim bridge was built so the firmware flies the aircraft's own aerodynamics. The detection model is being trained on a public coconut-disease image dataset; quantitative detection metrics and control-tuning of the bridge are the main items for Project-II. No hardware has been purchased.

*Keywords — autonomous UAV; VTOL quadplane; ArduPilot; coconut disease and pest detection; YOLO object detection; flight simulation.*

## TABLE OF CONTENTS

| Sl.No     | Contents                             | Page No.  |
|-----------|--------------------------------------|-----------|
|           | <b>Abstract</b>                      | <b>i</b>  |
| <b>1.</b> | <b>INTRODUCTION</b>                  | <b>1</b>  |
|           | 1.1 Background                       | <b>1</b>  |
|           | 1.2 Motivation                       | <b>1</b>  |
|           | 1.3 Scope of the Project             | <b>1</b>  |
| <b>2.</b> | <b>PROJECT DESCRIPTION AND GOALS</b> | <b>2</b>  |
|           | 2.1 Literature Review                | <b>2</b>  |
|           | 2.2 Research Gap                     | <b>2</b>  |
|           | 2.3 Objectives                       | <b>3</b>  |
|           | 2.4 Problem Statement                | <b>3</b>  |
|           | 2.5 Project Plan                     | <b>3</b>  |
| <b>3.</b> | <b>TECHNICAL SPECIFICATION</b>       | <b>5</b>  |
|           | 3.1 Requirements                     | <b>5</b>  |
|           | 3.1.1 Functional                     | <b>5</b>  |
|           | 3.1.2 Non-Functional                 | <b>5</b>  |
|           | 3.2 Feasibility Study                | <b>6</b>  |
|           | 3.2.1 Technical Feasibility          | <b>6</b>  |
|           | 3.2.2 Economic Feasibility           | <b>8</b>  |
|           | 3.2.3 Social Feasibility             | <b>8</b>  |
|           | 3.3 System Specification             | <b>8</b>  |
|           | 3.3.1 Hardware Specification         | <b>8</b>  |
|           | 3.3.2 Software Specification         | <b>9</b>  |
| <b>4.</b> | <b>DESIGN APPROACH AND DETAILS</b>   | <b>9</b>  |
|           | 4.1 System Architecture              | <b>9</b>  |
|           | 4.2 Design                           | <b>10</b> |
|           | 4.2.1 Data Flow Diagram              | <b>10</b> |
|           | 4.2.2 Use Case Diagram               | <b>11</b> |
|           | 4.2.3 Class Diagram                  | <b>12</b> |
|           | 4.2.4 Sequence Diagram               | <b>12</b> |
| <b>5.</b> | <b>REFERENCES</b>                    | <b>12</b> |

# 1. INTRODUCTION

## 1.1 Background

India is the world's largest coconut producer, and the crop underpins the rural economy of Tamil Nadu, Kerala, Karnataka and Andhra Pradesh. Yield is under constant pressure from diseases such as bud rot, stem bleeding, leaf rot and grey leaf spot, and from pests such as the rhinoceros beetle, red palm weevil and whitefly. Most of these are manageable if caught early and very difficult to reverse once they take hold.

Early detection depends on inspecting individual palms often. A coconut palm is 15 to 30 metres tall, so the crown and fronds where symptoms first appear are hard to see from the ground. Inspection is done by trained field staff walking the plantation, which is slow, subjective, and does not scale to plantations of thousands of trees.

Unmanned aerial vehicles are now common in precision agriculture, and deep-learning detectors have been shown to work on both ground and drone imagery of coconut palms. What has not been combined is an autonomous aerial platform that flies a real plantation route and feeds its imagery to a multi-class disease and pest detector. This project targets that combination.

## 1.2 Motivation

The motivation is the cost of late detection. Bud rot can kill a palm within months and spread to its neighbours; red palm weevil damage is often invisible until the crown collapses. Catching these early is worth far more than treating them late, but early detection needs frequent, whole-plantation inspection that manual methods cannot sustain.

An aerial survey changes what is practical. One flight can image every palm in a block, and an automated detector can screen those images consistently, flagging only the trees that need a closer look on the ground. A hybrid VTOL platform makes the flight itself practical: it takes off and lands vertically in the gaps between trees, needs no runway, and cruises efficiently enough to cover a whole plantation where a pure multirotor would need several battery swaps.

The project is also achievable by a student team. ArduPilot supplies a tested QuadPlane flight stack and a software-in-the-loop simulator, so the flight-control problem is one of configuration and validation, and the whole system can be proven in simulation before any hardware is bought.

## 1.3 Scope of the Project

In scope for Project-I: the quadplane airframe, propulsion, avionics and control-surface design; the ArduPilot QuadPlane parameter set and an autonomous waypoint mission demonstrated in SITL; a higher-fidelity aerodynamic model of the actual wing and tail, built from real airfoil data and verified in JSBSim and through an ArduPilot-to-JSBSim bridge; the YOLO-based disease and pest detection pipeline, including dataset selection, model training and evaluation; the Python route planner with a per-tree data store and dynamic pruning; and a telemetry dashboard.

Out of scope: physical procurement and construction of the aircraft; DGCA registration and outdoor flight; collection of an aerial-specific coconut image dataset; live obstacle avoidance during the confirm pass; and any treatment or spraying capability — the system detects and reports, it does not act on the tree. All Project-I results are from simulation and analysis; no hardware was flown.

## 2. PROJECT DESCRIPTION AND GOALS

### 2.1 Literature Review

The literature for this project spans two fields: deep-learning detection of coconut diseases and pests, and vertical take-off and landing (VTOL) fixed-wing UAV design for agriculture. Table 1 summarises the five works that most directly shape the approach taken here. The aerodynamic method also draws on standard aircraft-design texts (Raymer; Sadraey; Etkin and Reid) and on published low-Reynolds-number airfoil data.

| Paper                         | Objective                                                                                              | Methodology                                                      | Pros / Cons                                            | Findings                                                        |
|-------------------------------|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------------------------|
| Kadethankar et al. (2021)     | Detect red palm weevil infestation from drone imagery                                                  | CNN + Faster R-CNN on drone tree-crown crops                     | Real drone data; single-pest scope                     | 97.3 % infestation detection accuracy                           |
| Singh et al. (2021)           | Detect stem bleeding, leaf blight and red palm weevil                                                  | Segmentation + a custom 2-D CNN, compared with transfer learning | High accuracy; ground-level close-up images only       | 96.94 % validation accuracy                                     |
| Rohe et al. (2024)            | Count coconut palms in drone footage                                                                   | YOLOv7 with synthetic-image augmentation                         | Demonstrates YOLO on drone imagery; counting task only | mAP@0.5 improved 0.65 → 0.88                                    |
| Vidhanaarachchi et al. (2025) | Early diagnosis and severity of Weligama coconut leaf wilt disease and coconut caterpillar infestation | CNN + Mask R-CNN + YOLOv5 / v8 / v11                             | Adds severity grading; hand-captured leaf images       | Wilt 90 %, severity 97 %, YOLOv5 96.87 % mAP                    |
| Zhou et al. (2018)            | Review VTOL fixed-wing UAV designs for precision agriculture                                           | Literature review of VTOL configurations                         | Broad design coverage; not palm-specific               | Quadplane is the simplest hobbyist-supported VTOL configuration |

*Table 1. Reviewed literature.*

Recent coconut detection work falls into two groups. Ground-image studies (Singh et al., 2021; Vidhanaarachchi et al., 2025) reach very high accuracy — 96 to 97 per cent — on hand-collected close-up leaf and stem images, and the latter adds severity grading. Drone-image studies (Kadethankar et al., 2021; Rohe et al., 2024) show that YOLO-class and region-CNN detectors work on real aerial imagery, but each addresses a single narrow task — one pest, or tree counting. On the platform side, Zhou et al. (2018) review VTOL fixed-wing configurations for precision agriculture and identify the quadplane as the simplest configuration with mature hobbyist and open-source support.

### 2.2 Research Gap

Ground imagery versus aerial. The strongest coconut disease and pest models (Singh et al., 2021; Vidhanaarachchi et al., 2025) are trained and tested on hand-collected, ground-level, close-up images. None runs on imagery captured by an autonomous aircraft flying a real plantation route, where scale, viewing angle and image quality are different.

Single-task drone studies. Drone-based coconut work (Kadethankar et al., 2021; Rohe et al., 2024) confirms that YOLO-class detectors run on real drone imagery, but each targets one

narrow task — a single pest, or palm counting. None combines multi-class disease and pest detection with an actual flight mission.

Platform not paired with a mission. VTOL and quadplane UAV literature (Zhou et al., 2018) covers airframe trade-offs for agriculture in general, but is not paired with a re-visitable, dynamically pruned survey mission.

The combined gap. No identified work brings together (a) an ArduPilot quadplane VTOL platform sized for tight plantation spacing, (b) a two-tier scout-then-confirm route that drops cleared trees from future missions, and (c) onboard-recorded imagery fed to a YOLO-based multi-symptom detector. That combination is what this project targets.

## **2.3 Objectives**

The project has three objectives, each stated so that it can be checked against a concrete deliverable.

**O1.** Configure and validate ArduPilot QuadPlane firmware — a five-motor VTOL layout with two-elevon control — in SITL, demonstrating a stable arm, hover, transition and cruise sequence and a complete autonomous survey mission with all flight-phase checks passing.

**O2.** Train and evaluate a YOLO-based detection model for coconut disease and pest symptoms on a public dataset, targeting a measurable mAP@50 on a held-out validation split.

**O3.** Implement a Python route planner that computes an efficient waypoint order and dynamically prunes cleared trees, integrated with pymavlink for mission upload.

Sustainable Development Goals addressed: SDG 2 (Zero Hunger — reducing crop loss through earlier disease detection) and SDG 9 (Industry, Innovation and Infrastructure).

Expected outcome type: a validated design and simulation study, with a conference paper as the intended publication outcome.

## **2.4 Problem Statement**

Coconut diseases and pests cause large, recurring yield losses and are far cheaper to manage when caught early. Early detection depends on regularly inspecting individual palms, but the symptoms appear high on 15 to 30 metre trees and manual ground inspection across a plantation is slow, subjective and expert-dependent. Deep-learning detectors for coconut disease exist but are built for close-up ground images, and the drone-based coconut studies that do exist each solve only one narrow task.

This project addresses the design and simulation-based validation of an autonomous quadplane UAV that flies a planned per-tree route over a coconut plantation, records RGB imagery on board, and processes it after landing with a YOLO-based model that detects disease and pest symptoms — with a mission architecture that re-visits only the trees still flagged as needing attention.

## **2.5 Project Plan**

The Project-I work was organised in four overlapping streams: design and literature; flight software and SITL; aerodynamics and simulation; and the detection-pipeline and route-planning software. Figure 1 shows the schedule. The design decisions and the ArduPilot SITL mission were completed first, the real aerodynamic analysis and the JSBSim model followed and fed back two design corrections, and the ArduPilot-to-JSBSim bridge integrated the two. Detection-model training, bridge control tuning and the dashboard are scheduled after Review 2 and are shown hatched.

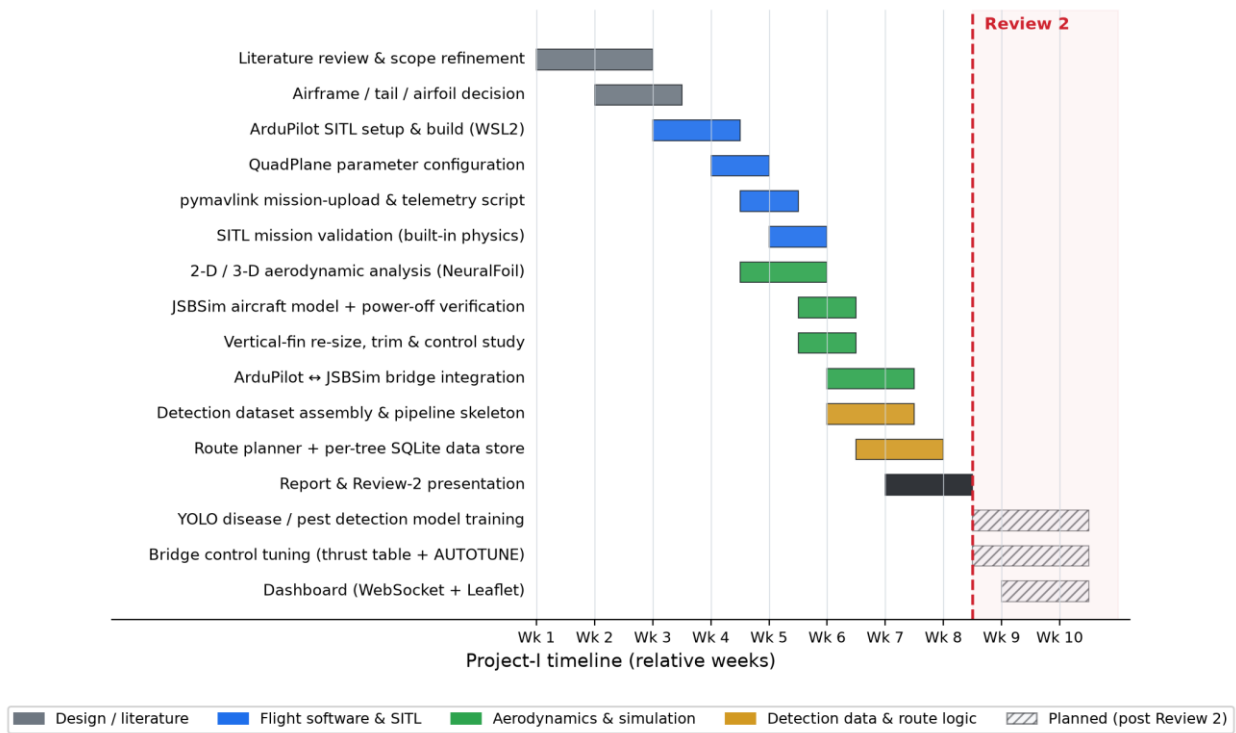


Fig. 1. Gantt chart

## 3. TECHNICAL SPECIFICATION

### 3.1 Requirements

#### 3.1.1 *Functional*

The functional requirements below define what the complete system must do; those marked for Project-II are specified now but demonstrated later.

**FR1. Autonomous mission execution** — The system shall autonomously execute a pre-computed multi-waypoint mission through ArduPilot QuadPlane, with no manual RC input.

**FR2. Waypoint imagery capture** — The payload shall capture RGB imagery of the palm at each active waypoint during the flight.

**FR3. Disease and pest detection** — The ground pipeline shall detect and classify visual disease and pest symptoms from the captured imagery using a YOLO-based model.

**FR4. Per-tree record** — The system shall maintain a persistent per-tree record — position, last-visit timestamp, detection confidence, and active / cleared status.

**FR5. Route recomputation with pruning** — The planner shall recompute the visiting route before each mission, excluding the waypoints marked cleared.

**FR6. Ground-station dashboard** — The system shall provide a ground-station dashboard showing live telemetry and mission status.

#### 3.1.2 *Non-Functional*

**Reliability and redundancy.** Four-motor quad-X lift layout so that a single lift-motor failure leaves a controllable, if degraded, hover; ArduPilot Q\_ASSIST provides motor assistance if the wing approaches a stall during transition.

**Endurance.** Hybrid VTOL configuration targeting more than 90 minutes of survey endurance, against 30 to 45 minutes for an equivalent multirotor, so a plantation is covered in a single sortie.

**Latency tolerance.** Detection runs after landing, so there is no hard real-time inference requirement. The only hard real-time loop is the flight controller's own stabilisation and navigation, handled by ArduPilot.

**Safety.** Geofence and return-to-launch failsafes shall be configured; a manual RC override shall be available during any real test flight.

**Weight.** All-up weight shall remain within the DGCA Micro category, 250 g to 2 kg (design estimate 1.4 kg).

**Cost.** The hardware bill of materials shall stay within a student budget, targeted at 15,000 rupees or less.

**Simplicity and field repair.** No tilting mechanisms; a foam airframe with a carbon spar that can be repaired in the field; commodity components with common spares.

**Regulatory compliance.** The aircraft shall be registerable under the Drone Rules, 2021 before any outdoor flight.

## 3.2 Feasibility Study

### 3.2.1 Technical Feasibility

The technical feasibility of the flight system is supported by working simulation. A complete autonomous mission was flown in ArduPilot SITL: all 44 configured parameters loaded with no mismatch, the mission uploaded and read back correctly, and all five flight phases — VTOL take-off, transition to forward flight, the survey waypoints, VTOL landing and automatic disarm — completed, reaching 37.8 m maximum altitude and 18.3 m/s maximum airspeed over a 61.8 s flight with no crash.

The aerodynamic design is backed by real analysis. The SD7037 wing and NACA 0009 tail sections were run through NeuralFoil (an XFOIL surrogate) at the cruise Reynolds number of about  $1.9 \times 10^5$  and reduced to a three-dimensional aircraft model with AeroSandbox, cross-checked against a vortex-lattice solution. That analysis produced two design corrections — wing incidence reduced from  $2^\circ$  to  $1^\circ$  and cruise speed from 16 to 12 m/s — and the vertical fin was enlarged from about 210 to 340 cm<sup>2</sup> (a 62 % increase) to raise directional stiffness from an inadequate  $Cn_\beta \approx 0.02$  /rad to the 0.07 /rad target. The corrected configuration has a 27 % MAC static margin and a cruise lift-to-drag ratio of about 13.5. Elevon-only pitch and roll control was checked as adequate: a 2.6 g pull-up (wing-stall limited) and about 657°/s roll rate at full elevon.

The corrected model was ported to JSBSim and passed all eight power-off flight-dynamics checks: a trimmed glide at  $L/D \approx 12$ , a pitch disturbance decaying from  $1.26^\circ$  to  $0.32^\circ$ , a bounded phugoid, and a  $4^\circ$  sideslip washing out to  $0.04^\circ$  — the enlarged fin working. The ArduPilot flight stack was then flown against this JSBSim model through a bridge running at 400 frames per second in lockstep with no numerical divergence. Integrating that bridge also caught a real bug: the elevon roll and pitch commands were sign-inverted, which would have caused loss of control on a first real flight, and was found and fixed in simulation.

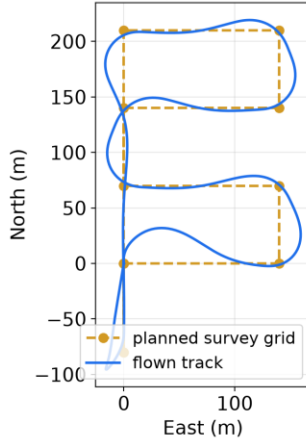
Open technical items, stated plainly. Running the full mission on the higher-fidelity aerodynamics completes all phases but shows a survey-phase instability — airspeed swinging between about 2 and 25 m/s against a 12 m/s target, the aircraft porpoising, and the VTOL landing overshooting by roughly 55 to 75 m before recapture. The cause is identified: survey waypoints spaced tighter than the fixed-wing turn radius, a placeholder propulsion model (the detailed motor model is numerically unstable near hover), and control gains that have not yet been auto-tuned. Each has a defined fix — a measured thrust table, ArduPilot AUTOTUNE, and transition and waypoint-spacing tuning. Hover and transition dynamics on the bridge are therefore not yet flight-validated.

The detection model is in training on the public Coconut Tree Disease Dataset (Thite et al., 2023). Quantitative detection metrics (mAP@50 on a held-out split) are pending and will be reported in Project-II; the known domain gap is that this dataset is ground-captured rather than aerial, which an aerial-specific dataset will need to close before deployment.

All the software used — ArduPilot, SITL, JSBSim, NeuralFoil, AeroSandbox, pymavlink, YOLO / PyTorch, FreeCAD — is free and open-source, and the analysis, simulation and model training run on a laptop, so there is no tooling or infrastructure barrier.

### ArduPilot SITL — full mission, all flight-phase checks PASS

Autonomous survey grid — planned vs flown



VTOL takeoff → transition → cruise → VTOL land

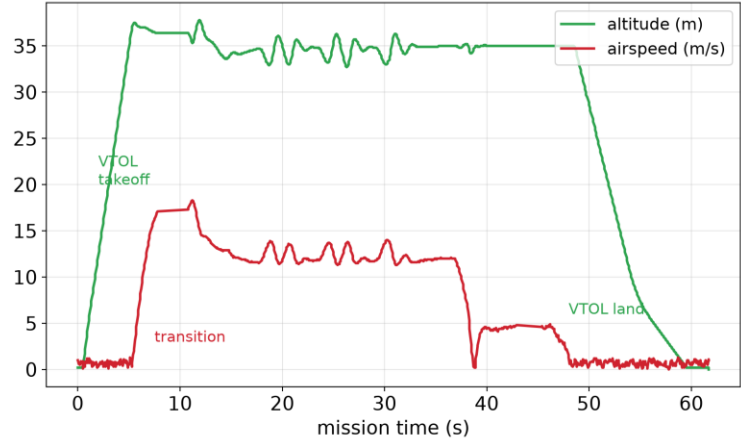


Fig. 5. ArduPilot SITL — full autonomous mission; every flight-phase check passes.

### Real 3-D aerodynamics — SD7037 wing / NACA 0009 tail (NeuralFoil + AeroSandbox, $Re \approx 191k$ )

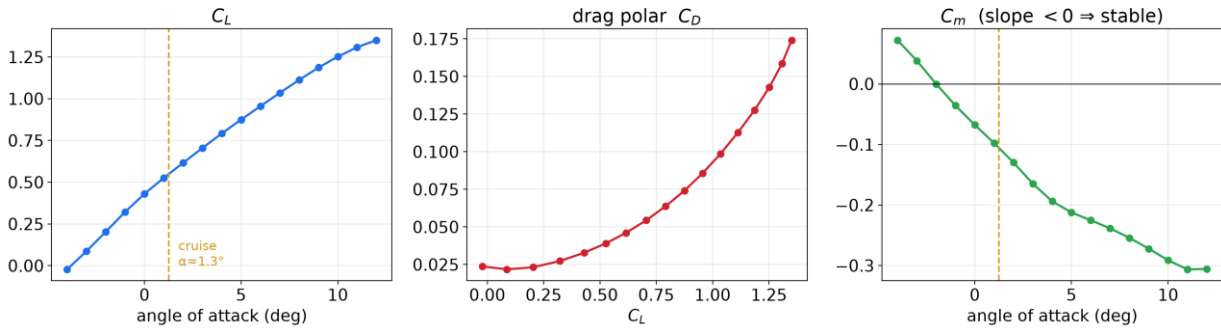
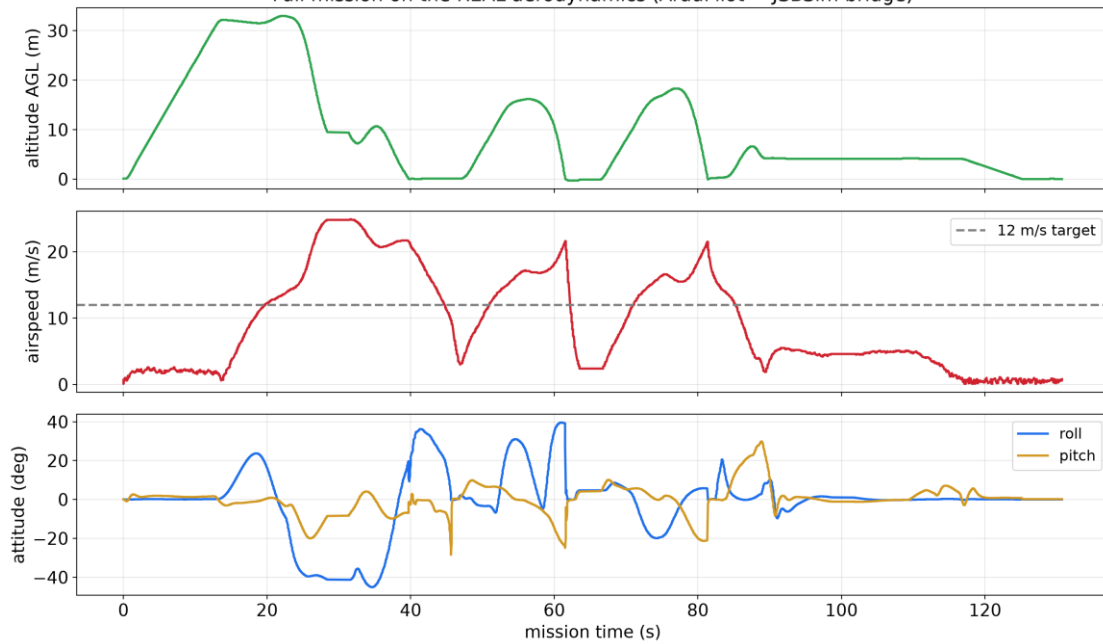


Fig. 6. Real 3-D aerodynamic coefficients of the SD7037 wing / NACA 0009 tail (NeuralFoil + AeroSandbox).

### Survey-phase instability — diagnosed, fix in progress

Full mission on the REAL aerodynamics (ArduPilot ↔ JSBSim bridge)



*Fig. 7. Full mission on the higher-fidelity aerodynamics — the survey-phase instability under diagnosis.*

| <b>Metric</b>                                  | <b>Result (simulation / analysis)</b>                               |
|------------------------------------------------|---------------------------------------------------------------------|
| SITL mission — flight-phase checks             | all pass (take-off, transition, grid, land, disarm)                 |
| Power-off flight-dynamics checks (JSBSim)      | 8 / 8 pass                                                          |
| Cruise glide L/D (power-off test)              | $\approx 12$                                                        |
| Static margin / trim $\alpha$ / tail incidence | 27 % MAC / $1.3^\circ$ / $-1.8^\circ$                               |
| Elevon pitch / roll authority                  | 2.6 g pull-up; $\approx 657^\circ/\text{s}$ roll rate (full elevon) |
| Vertical-fin resize / yaw stiffness $C_n\beta$ | $210 \rightarrow 340 \text{ cm}^2$ / $+0.02 \rightarrow +0.07$ /rad |
| Wing incidence / cruise speed (post-analysis)  | $2^\circ \rightarrow 1^\circ$ / $16 \rightarrow 12 \text{ m/s}$     |
| ArduPilot $\leftrightarrow$ JSBSim bridge      | closed, 400 fps lockstep; elevon sign bug fixed                     |
| YOLO disease / pest model                      | in training on public dataset — mAP pending (Project-II)            |

*Table 2. Key simulation and analysis results to date.*

### **3.2.2 Economic Feasibility**

The project is being validated in simulation at zero hardware cost; the figures below are the planned build cost, not spent in Project-I.

The new-purchase bill of materials is estimated at 8,200 to 10,900 rupees. Within that, the motor, ESC and propeller set is a confirmed bulk price of about 4,370 rupees and the two elevon servos about 538 rupees; the flight controller, time-of-flight sensor and camera are estimates. Including the foam and battery already on hand, the full build is about 9,500 to 14,200 rupees, inside the 15,000-rupee target.

Two deliberate cost decisions shaped this. A thermal camera (about 5,000 rupees on its own, the entire original budget) was dropped because the detection task is colour-based. Using the same A2212 motor for lift and forward thrust simplifies sourcing and spares. The return on investment is framed not in rupees but in avoided crop loss from earlier disease detection and in the field-inspection hours saved.

### **3.2.3 Social Feasibility**

The primary social benefit is earlier detection of disease and pest outbreaks, which reduces crop loss and can reduce the amount of pesticide used by making treatment targeted rather than blanket. The system is designed to assist plantation field staff, not replace them: it screens every tree and hands back a short list for a human to confirm and act on.

The privacy footprint is small. The aircraft carries an RGB camera only, no thermal sensor, imagery is confined to the operator's own plantation and processed locally on a laptop. Before any outdoor flight the aircraft must be registered under the Drone Rules, 2021 and flown within the Micro-category altitude limit, which comfortably clears the 15 to 30 m palm height the mission needs.

## **3.3 System Specification**

The hardware and software actually required by the project are listed below.

### **3.3.1 Hardware Specification**

| Item              | Specification                                                                                                                                                                                           |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Airframe          | Custom quadplane — 40–50 cm pod, 1.3 m wing span (0.30 m <sup>2</sup> area), EPP/EPO foam with a carbon spar; SD7037 wing section, NACA 0009 fixed tail, twin vertical fins (170 cm <sup>2</sup> each). |
| Lift motors       | 4 × A2212 2200KV brushless, standard quad-X layout (two front, two rear).                                                                                                                               |
| Forward motor     | 1 × A2212 2200KV brushless, pusher-mounted (same part as the lift motors for common spares).                                                                                                            |
| ESCs              | 5 × SimonK 30 A.                                                                                                                                                                                        |
| Propellers        | 1045 (10 × 4.5) — 2 CW + 2 CCW for lift, 1 for forward thrust.                                                                                                                                          |
| Flight controller | Darkmatter Brahma F4 MK-III (STM32F405), official ArduPilot support, 9 PWM outputs (5 motors + 2 servos + headroom).                                                                                    |
| Control surfaces  | 2 × elevons (combined pitch and roll), driven by TowerPro MG90S digital metal-gear servos.                                                                                                              |
| Ranging sensor    | VL53L1X time-of-flight — altitude and close-range obstacle sensing.                                                                                                                                     |
| Camera            | ESP32-CAM-class RGB module — records disease and pest imagery of each palm to onboard storage.                                                                                                          |
| Battery           | 3S/4S LiPo (already on hand); powers all five motors and the avionics.                                                                                                                                  |
| Ground station    | Laptop running the mission-upload script, the YOLO detection pipeline and the dashboard — no onboard companion computer.                                                                                |

Table 3. Hardware specification (planned build; not purchased in Project-I).

### 3.3.2 Software Specification

| Layer                   | Tool / approach                                                                                                                                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Flight firmware         | ArduPilot QuadPlane — configured through parameters, not forked.                                                                                                                      |
| Flight simulation       | ArduPilot SITL (built-in QuadPlane physics) for the mission demonstration.                                                                                                            |
| Flight-dynamics model   | JSBSim 6-DOF with a custom CoconutQuadplane aircraft model; ArduPilot-to-JSBSim lockstep bridge over UDP.                                                                             |
| Aerodynamic analysis    | NeuralFoil (2-D sections) and AeroSandbox AeroBuildup / VLM (3-D aircraft); Python, run on a laptop.                                                                                  |
| Ground-station link     | pymavlink / DroneKit (MAVLink) — mission upload and telemetry logging.                                                                                                                |
| Route planner           | Custom Python — visiting order over active waypoints, dynamic pruning, MAVLink mission export.                                                                                        |
| Data store              | SQLite — per-tree status, confidence, last visit, active / cleared flag.                                                                                                              |
| Detection model         | YOLO-based disease and pest symptom detection (with a second model for frond health); trained on the public Coconut Tree Disease Dataset (Thite et al., 2023); PyTorch / Ultralytics. |
| Dashboard               | WebSocket backend bridging MAVLink telemetry; Leaflet map front-end with battery, mission time and detections.                                                                        |
| CAD                     | FreeCAD with the AirPlaneDesign workbench — wing ribs and the parametric vertical-fin model.                                                                                          |
| Development environment | Ubuntu on WSL2; Python 3; Git.                                                                                                                                                        |

Table 4. Software stack.

## 4. DESIGN APPROACH AND DETAILS

### 4.1 System Architecture

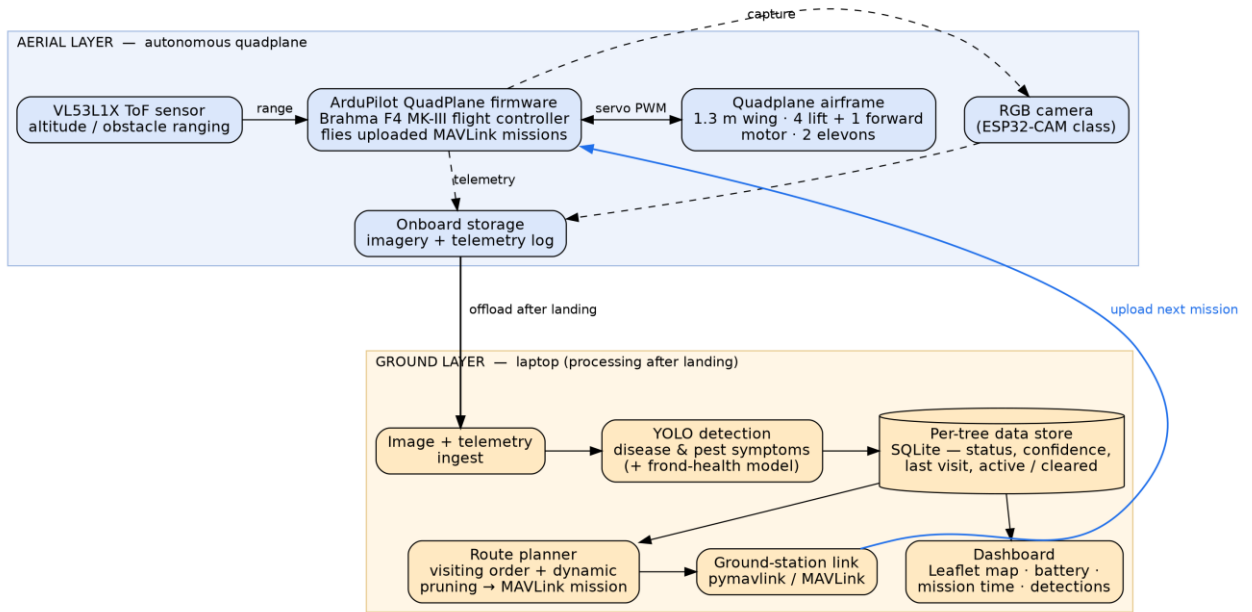


Fig. 2. System architecture — aerial tier and ground (laptop) tier.

The system has two physical tiers, shown in Figure 2, and is built as four modules: flight control and simulation, detection, route planning and data store, and ground station and dashboard.

On the aircraft, the ArduPilot flight controller flies the uploaded mission, driving the elevons and the five motors and reading its IMU and GPS; the RGB camera records palm imagery to onboard storage on a capture trigger, the VL53L1X supplies range for altitude hold, and the telemetry log is stored alongside the imagery. There is no onboard companion computer — imagery is recorded, not streamed.

On the ground, everything runs on a laptop and everything downstream of image capture happens after the aircraft lands. Imagery and the telemetry log are offloaded, the YOLO model scores each tree for disease and pest symptoms, and the results update the per-tree SQLite store. The route planner reads the active trees from that store, computes a visiting order, prunes the cleared trees, and exports the next MAVLink mission, which the ground-station link uploads to the aircraft. The dashboard reads the same store and the live telemetry.

Recording on board and processing after landing — rather than streaming live — is a deliberate choice: the survey is not time-critical, and it removes the Wi-Fi range and canopy-dropout problem that live streaming under a plantation canopy would create.

## 4.2 Design

### 4.2.1 Data Flow Diagram

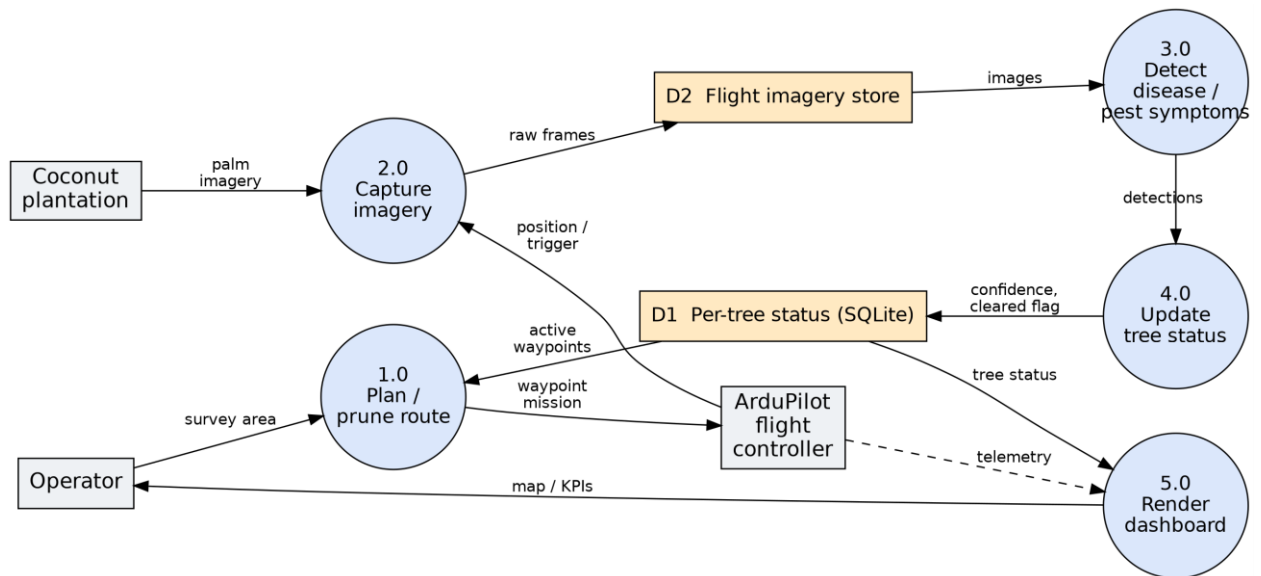


Fig. 3. Level-1 data flow diagram.

Figure 3 is the level-1 data flow diagram. The Operator supplies the survey area; process 1.0 plans and prunes the route using the current per-tree status (D1) and issues a waypoint mission to the ArduPilot flight controller. During the flight, process 2.0 captures palm imagery (triggered on position) into the imagery store (D2). After landing, process 3.0 runs the YOLO disease and pest detection on that imagery, process 4.0 turns the detections into per-tree status updates written back to D1, and process 5.0 renders the dashboard from D1 and the telemetry, returning the map and key indicators to the Operator.

#### 4.2.2 Use Case Diagram

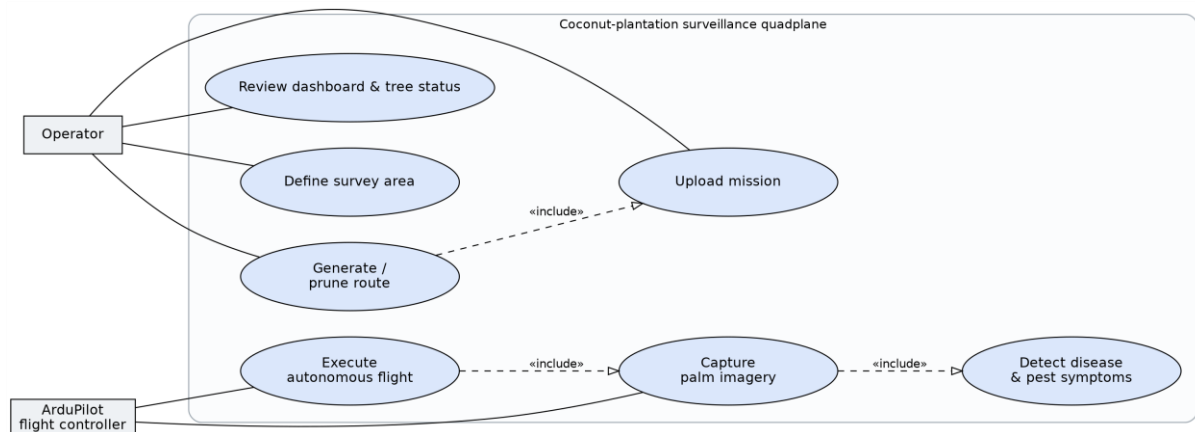


Fig. 4. Use-case diagram.

Figure 4 is the use-case diagram. There are two actors: the Operator, who defines the survey area, triggers route generation and mission upload, and reviews the dashboard and per-tree status; and the ArduPilot flight controller, which executes the autonomous flight and drives imagery capture. Route generation includes mission upload; executing the flight includes capturing imagery, which in turn includes running the disease and pest detection step on the recorded images after landing.

**4.2.3 Class Diagram (optional)** — not included in this report. The software components (route planner, per-tree data store, detection pipeline) are at skeleton stage; a class model will be produced in Project-II once the interfaces between them are stable.

**4.2.4 Sequence Diagram (optional)** — not included in this report, for the same reason. The mission-level interaction order is captured by the data flow diagram (Figure 3) and the use-case include-relationships for now.

## 5. REFERENCES

### **Journals:** <IEEE Format>

- [1] P. Singh, A. Verma, and J. S. R. Alex, “Disease and pest infection detection in coconut tree through deep learning techniques,” *Computers and Electronics in Agriculture*, vol. 182, art. no. 105986, Mar. 2021.
- [2] S. Vidhanaarachchi, J. L. Wijekoon, W. A. S. P. Abeysiriwardhana, and M. Wijesundara, “Early diagnosis and severity assessment of Weligama coconut leaf wilt disease and coconut caterpillar infestation using deep-learning-based image processing techniques,” *IEEE Access*, 2025.
- [3] S. Thite, Y. Suryawanshi, K. Patil, and P. Chumchu, “Coconut (*Cocos nucifera*) tree disease dataset: a dataset for disease detection and classification for machine learning applications,” *Data in Brief*, vol. 51, art. no. 109690, Oct. 2023.
- [4] M. Zhou, Z. Zhou, L. Liu, J. Huang, and Z. Lyu, “Review of vertical take-off and landing fixed-wing UAV and its application prospect in precision agriculture,” *International Journal of Precision Agricultural Aviation*, 2018.

### **Conference:** <IEEE Format>

- [5] A. Kadethankar, N. Sinha, A. Burman, and V. Hegde, “Deep-learning-based detection of rhinoceros beetle infestation in coconut trees using drone imagery,” in *Computer Vision and Image Processing (CVIP 2020)*, Springer, 2021, pp. 463–474.
- [6] J. S. Berndt, “JSBSim: an open source flight dynamics model in C++,” in *Proc. AIAA Modeling and Simulation Technologies Conf.*, Providence, RI, USA, 2004, AIAA 2004-4923.

### **Weblinks:**

- [7] T. Rohe, D. Hein, T. Reder, M. Grutzmann, F. Rueping, and H. Bieker, “Coconut palm tree counting on drone images with deep object detection and synthetic training data,” *arXiv:2412.11949*, 2024.
- [8] ArduPilot Development Team, “ArduPilot documentation — QuadPlane and SITL,” <https://ardupilot.org/plane/> (accessed Sep. 2026).
- [9] P. Sharpe, “NeuralFoil: airfoil aerodynamics analysis,” <https://github.com/peterdsharpe/NeuralFoil> (accessed Sep. 2026).
- [10] M. S. Selig, “UIUC Airfoil Coordinates Database,” University of Illinois at Urbana-Champaign, [https://m-selig.ae.illinois.edu/ads/coord\\_database.html](https://m-selig.ae.illinois.edu/ads/coord_database.html) (accessed Sep. 2026).
- [11] Ministry of Civil Aviation, Government of India, “The Drone Rules, 2021,” <https://www.civilaviation.gov.in/> (accessed Sep. 2026).

**Book:**

[12] D. P. Raymer, Aircraft Design: A Conceptual Approach, 6th ed. Reston, VA, USA: AIAA Education Series, 2018.

[13] M. H. Sadraey, Aircraft Design: A Systems Engineering Approach. Chichester, UK: Wiley, 2012.

[14] B. Etkin and L. D. Reid, Dynamics of Flight: Stability and Control, 3rd ed. New York, NY, USA: Wiley, 1996.